

Lecture 1: Working at the Unix Command Line

Big Data Algorithms

Plan for the session

Time	Topic	Commands
0–10	Anatomy of a command, getting help	<code>man</code> , <code>--help</code> , Tab, history, Ctrl+C, <code>clear</code>
10–25	Moving around	<code>pwd</code> , <code>ls</code> , <code>cd</code> , <code>which</code> , <code>echo</code>
25–45	Managing files	<code>mkdir</code> , <code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>find</code> , <code>chmod</code> , <code>sudo</code>
45–55	Editing in the terminal	<code>vi</code> , <code>nano</code>
55–65	<i>Break</i>	
65–85	Looking at data without opening it	<code>cat</code> , <code>less</code> , <code>head</code> , <code>tail</code> , <code>wc</code> , <code>du</code> , <code>df</code> , <code>time</code>
85–100	Redirection and pipes	<code>></code> , <code>>></code> , <code> </code> , <code>grep</code> , <code>cut</code> , <code>sort</code> , <code>uniq</code>
100–115	Processes	<code>ps</code> , <code>htop</code> , <code>top</code> , <code>kill</code> , <code>&</code> , <code>jobs</code> , <code>fg</code>
115–120	Wrap-up	

Mac users

Almost everything here works the same in the Mac Terminal. However, the Linux command `free` doesn't exist on a Mac (use `top` instead).

Setup: practice files

Copy and paste this block into your terminal. It creates a folder for today and two practice files: `big.txt` (the numbers 1 to 10 million, about 76 MB) and `data.csv` (one million rows of made-up data, about 15 MB). Don't worry about understanding the `awk` command yet; we'll get to it later in the course.

```
mkdir -p ~/bigdata/lec01
cd ~/bigdata/lec01
seq 1 10000000 > big.txt
awk 'BEGIN { srand(1); print "id,group,value";
  for (i = 1; i <= 1000000; i++)
    printf "%d,%s,%.2f\n", i, (rand() < 0.5 ? "A" : "B"), 100 * rand() }' > data.csv
```

Anatomy of a command, getting help

A command has three parts: the **command** itself, **options** (also called flags) that change its behavior, and **arguments** that tell it what to act on.

```
ls -l -h ~/bigdata
# ~ ^^^^^^ ^^^^^^^^^
# | options  argument
# command
```

Short options can be combined: `ls -lh` is the same as `ls -l -h`.

Getting help:

```
man ls      # the full manual page (press q to quit)
ls --help   # a shorter summary (GNU/Linux tools)
```

Keyboard habits that save hours:

Key	What it does
Tab	Completes file and command names. Press twice to see options.
↑ / ↓	Scroll through previous commands
Ctrl+R	Search your command history
Ctrl+C	Stop the running command
Ctrl+L or <code>clear</code>	Clear the screen

```
history      # list your recent commands
```

Moving around

The file system is a tree. You are always “in” one directory, called the **working directory**.

```
pwd          # print working directory: where am I?
ls           # list what's here
ls -l        # long format: permissions, owner, size, date
ls -lh       # ... with human-readable sizes
ls -a        # include hidden files (names starting with .)
cd ~/bigdata # go to a directory
cd ..        # go up one level
cd           # go home (same as cd ~)
cd -         # go back to where you just were
```

Paths:

- An **absolute path** starts from the root, /: /home/adam/bigdata/lec01
- A **relative path** starts from where you are: lec01/data.csv
- ~ means your home directory, . means “here”, .. means “one level up”

Two more commands for finding your way:

```
which python3    # where is this program installed?
echo $HOME        # print a variable: your home directory
echo $PATH        # the directories searched for programs
```

 Try it

Starting from your home directory, get into ~/bigdata/lec01 using only relative paths, then get back home in one command.

Managing files

```
mkdir scratch    # make a directory
mkdir -p a/b/c    # make nested directories in one step
touch notes.txt   # create an empty file (or update its timestamp)
cp notes.txt notes2.txt # copy a file
cp -r a a_copy    # copy a directory (-r = recursive)
mv notes2.txt scratch/ # move a file
```

```
mv scratch old_scratch      # rename (moving to a new name)
rm notes.txt                # delete a file
rm -r old_scratch           # delete a directory and everything in it
rm -i *.txt                 # ask before deleting each file
```

There is no trash can

`rm` deletes permanently. There is no undo. Be especially careful with `rm -r` and with wildcards. Before running `rm` on a pattern, run `ls` with the same pattern to see what it will match.

Wildcards match file names:

```
ls *.csv                    # every file ending in .csv
ls file_?.txt               # ? matches exactly one character
```

Finding files:

```
find ~/bigdata -name "*.csv" # all csv files under ~/bigdata
find . -size +50M            # files bigger than 50 MB under here
```

Permissions. In the output of `ls -l`, the first column looks like `-rwxr-xr--`: read, write, and execute permissions for the owner, the group, and everyone else.

```
chmod +x myscript.sh      # make a script executable
sudo apt install htop      # run one command as administrator
```

`sudo` asks for your Linux password. Use it only when a command needs it, typically for installing software.

Editing in the terminal

On a remote server there is no graphical editor, so you need to be able to edit a file in the terminal. `vi` is installed on virtually every Unix system. It has **modes**, which is what confuses everyone at first.

```
vi notes.txt
```

The survival kit:

Keys	What it does
i	Enter insert mode: now you can type
Esc	Leave insert mode
:w Enter	Save
:q Enter	Quit
:wq Enter	Save and quit
:q! Enter	Quit without saving
/word Enter	Search for “word” (n for next match)
dd	Delete the current line
u	Undo

If you’re ever stuck in vi, press Esc a few times, then type :q! and Enter.

nano is a simpler editor with its commands listed at the bottom of the screen (^ means Ctrl). It’s fine for quick edits:

```
nano notes.txt      # Ctrl+O to save, Ctrl+X to exit
```

💡 Practice later

vimtutor is a 30-minute interactive vi tutorial. On Ubuntu, install it with `sudo apt install vim`.

Looking at data without opening it

This is the most important section for big data. A file can be far larger than what you want to (or can) load into memory, but you can still look at it.

```
cat data.csv          # print the whole file (bad idea for big files!)
less big.txt          # scroll through a file (q quits, / searches)
head data.csv         # first 10 lines
head -n 3 data.csv    # first 3 lines
tail data.csv         # last 10 lines
tail -f logfile.txt   # keep watching a file as it grows (Ctrl+C to stop)
wc -l data.csv        # count lines
```

💡 Try it

Run `cat big.txt`. Then press `Ctrl+C`. Now run `head big.txt` and `tail big.txt`. Which ones are instant, and why?

How much space are things taking?

```
du -sh ~/bigdata      # total size of a directory
du -sh *              # size of each item here
df -h ~               # free space on the disk holding your home directory
```

How long does a command take? Put `time` in front of it:

```
time wc -l big.txt
```

`real` is the wall-clock time. We'll use `time` all semester to compare methods.

Redirection and pipes

By default, output goes to the screen. You can send it to a file instead:

```
echo "hello" > greeting.txt    # write to a file (replaces its contents!)
echo "again" >> greeting.txt    # append to a file
head -n 1000 data.csv > sample.csv
```

The **pipe**, `|`, sends the output of one command into the next. Each command does one small job, and you chain them:

```
grep ",A," data.csv | head      # rows in group A, first 10
grep -c ",A," data.csv         # count of rows in group A
cut -d, -f2 data.csv | head     # just the 2nd column
cut -d, -f2 data.csv | tail -n +2 | sort | uniq -c # count rows per group
```

Read the last line left to right: take column 2, drop the header line, sort it, then count identical neighbors.

Why this matters for big data

Pipes process data as a **stream**: each command reads a line, handles it, and passes it on. None of these commands loads the whole file into memory, which is why they work on files much bigger than your RAM. This is the idea behind the one-pass algorithms we'll study later.

Try it

How many rows of `data.csv` have a value starting with 99.? (Hint: cut the third column, then `grep` with `"^99\"`, then count.)

Processes

Every running program is a **process** with an ID number (PID).

```
ps                # your processes in this terminal
ps aux | head     # every process on the machine
htop              # live view of CPU and memory (q quits)
top               # the older version, available everywhere
```

Demo: watch a process eat CPU. Run this in one terminal, and `htop` in another:

```
yes > /dev/null &    # a useless program that uses 100% of one core
```

The `&` runs it in the **background**, so you get your prompt back. Now manage it:

```
jobs              # list background jobs in this terminal
fg                # bring it to the foreground (then Ctrl+C stops it)
kill %1           # or: stop background job number 1
pkill yes         # or: stop every process named yes
kill 12345        # or: stop a process by its PID
kill -9 12345     # last resort: force it
```

Demo: watch a process use memory. With `htop` open, run:

```
python3 -c "x = 'a' * 10**9; input('Press Enter to quit')"
```

Watch the memory bar jump by about 1 GB. Press Enter and watch it drop. This is what “fits in RAM” means in practice.

Wrap-up

Cheat sheet

Command	What it does
<code>pwd, ls, cd</code>	Where am I, what's here, go somewhere
<code>mkdir, touch</code>	Make a directory, make an empty file
<code>cp, mv, rm</code>	Copy, move/rename, delete (permanently!)
<code>find</code>	Search for files by name, size, etc.
<code>chmod, sudo</code>	Change permissions, run as administrator
<code>vi, nano</code>	Edit files in the terminal
<code>cat, less</code>	Print a file, scroll through a file
<code>head, tail, wc</code>	First lines, last lines, count lines
<code>du, df</code>	Space used by files, free space on disk
<code>time</code>	How long a command takes
<code>>, >>, </code>	Write to file, append to file, pipe to next command
<code>grep, cut, sort, uniq</code>	Filter lines, pick columns, sort, count duplicates
<code>ps, htop, top</code>	See running processes
<code>&, jobs, fg, kill</code>	Background, list, foreground, stop processes
<code>man, --help</code>	Get help

Check yourself

These are the kinds of questions you should be able to answer on an exam, without a computer or AI.

1. What's the difference between an absolute and a relative path? Give an example of each.
2. You're in `~/bigdata/lec01`. What does `cd ../../` do?
3. What's the difference between `>` and `>>`?
4. You opened a file in `vi` and typed text you don't want to keep. How do you get out without saving?
5. Why is `head` instant on a 10 GB file while `cat` is not?
6. What does `cut -d, -f2 data.csv | sort | uniq -c` produce?
7. Your laptop's fan is suddenly loud. How do you find out which process is responsible, and how do you stop it?
8. Why is `rm -r *` dangerous?

Before next class

- Make sure your terminal works: Ubuntu in WSL on Windows, or Terminal on a Mac.

- Install `htop` and `git` if you don't have them.
- Do 15 minutes of `vimtutor`.